

BIG DATA ANALYTICS · LAB MANUAL

Hadoop & Kafka Practical Components

Five hands-on labs, fully containerised with Docker

Course: Big Data Analytics (Units 2–5)

Environment: Docker Desktop on Windows 10/11

Languages: Java and Python (Kafka labs give both)

Prepared for: Dept. of CSE

WORD COUNT · HDFS JAVA API · KAFKA PRODUCER/CONSUMER · CONSUMER GROUPS ·
AVRO

Contents

A Before You Begin — Docker Desktop Setup

Installing Docker Desktop · verifying it works · how these labs are organised

1 Word Count in MapReduce

Units 2–3 · Hadoop in local mode · mapper, reducer, combiner

2 HDFS File Operations with the Java FileSystem API

Unit 2 · create, seek, listStatus, globStatus on real HDFS

3 Kafka Producer & Consumer

Units 4–5 · keyed messages, partitions, the poll loop · Java + Python

4 Consumer-Group Scaling & Rebalance

Unit 5 · watch partitions split and rebalance live · Java + Python

5 Avro Serialization Round-Trip

Units 2 & 4 · schema-based produce and consume · Java + Python

HOW EACH LAB IS STRUCTURED

Every lab follows the same five parts so you always know where you are: **Aim** (what you will build and why) · **Requirements** (what must be running) · **Setup** (the Docker commands) · **The Program** (code with a line-by-line explanation) · **Execution & Expected Output** (exact commands and what you should see).

A · Before You Begin — Docker Desktop Setup

Everything in this manual runs inside **Docker containers**. You do *not* install Hadoop or Kafka on Windows directly — instead Docker runs them in isolated, disposable environments. This means every student gets exactly the same setup, and you can throw a broken environment away and start fresh in seconds.

System requirements

Item	Minimum	Notes
OS	Windows 10 (64-bit, build 19044+) or Windows 11	Pro, Enterprise, or Home all work
RAM	8 GB (16 GB recommended)	Kafka + Hadoop containers are memory-hungry
Disk	10 GB free	For Docker images
CPU	64-bit with virtualization (SLAT)	Must be enabled in BIOS — see below
Feature	WSL 2	Docker Desktop installs this for you

VIRTUALIZATION MUST BE ON

Docker Desktop needs hardware virtualization. Open **Task Manager** → **Performance** → **CPU** and check that *Virtualization: Enabled* appears at the bottom right. If it says *Disabled*, reboot into BIOS/UEFI and enable *Intel VT-x* or *AMD-V*.

1 Install Docker Desktop

Download the installer from docker.com/products/docker-desktop and run it. When asked, keep “**Use WSL 2 instead of Hyper-V**” ticked. Reboot when it finishes, then launch Docker Desktop and accept the terms. Wait until the whale icon in the system tray stops animating — that means the engine is running.

2 Verify it works

Open **PowerShell** (press Win, type *PowerShell*, press Enter) and run:

```
PS> docker --version
# Docker version 27.x.x, build ...

PS> docker run hello-world
```

The `hello-world` container downloads a tiny image and prints a success message. If you see “*Hello from Docker!*”, your environment is ready.

CHECKPOINT

Both commands succeeded and you saw the “Hello from Docker!” message. If not, make sure the Docker Desktop whale icon is solid (not animating) and virtualization is enabled, then try again.

3 Create a working folder

Make one folder to hold all your lab files. In PowerShell:

```
PS> mkdir C:\bda-labs  
PS> cd C:\bda-labs
```

You will create sub-folders inside this for each lab as you go.

THREE COMMANDS YOU WILL USE CONSTANTLY

docker ps — list running containers · **docker exec -it <name> bash** — open a shell *inside* a running container · **docker rm -f <name>** — force-remove a container when you are done. Typing **exit** leaves a container's shell without stopping it.

Lab 1 · Word Count in MapReduce

The “Hello World” of Hadoop — see the map, shuffle, and reduce stages run

UNITS 2–3

JAVA

Aim

Write and run the classic MapReduce program: count how many times each word appears across a set of text files. You will see the exact stages from your Unit 3 slides — the **map** task emitting `(word, 1)` pairs, the **shuffle and sort** grouping them by key, and the **reduce** task summing the counts. You will then add a **combiner** and observe the reduced shuffle traffic.

Requirements

- Docker Desktop running (Section A complete)
- The official `apache/hadoop` image — Docker downloads it automatically
- No Java installed on Windows — the container already has it

WHY LOCAL MODE?

Hadoop can run as a full multi-node cluster, but for learning the *logic* of MapReduce we run it in **local (standalone) mode** inside one container. The job still goes through map → shuffle → reduce exactly as it would on a cluster — you just see it all in one place, in the logs.

Setup

1 Start a Hadoop container

```
PS> cd C:\bda-labs
PS> mkdir lab1-wordcount
PS> cd lab1-wordcount

# start a Hadoop container that stays running, named "hadoop"
PS> docker run -dit --name hadoop apache/hadoop:3.3.6 bash
```

The `-dit` flags mean *detached, interactive, with a terminal*, so the container keeps running in the background. `--name hadoop` gives it a name we can refer to.

2 Open a shell inside the container

```
PS> docker exec -it hadoop bash

# your prompt changes — you are now INSIDE the container:
hadoop@a1b2c3:~$
```

INSIDE VS. OUTSIDE

When the prompt shows `hadoop@...:~$` you are *inside* the container. When it shows `PS>` you are back in Windows PowerShell. The next steps all run *inside* the container unless stated otherwise.

The Program

MapReduce Word Count has three pieces: a **Mapper**, a **Reducer**, and a **Driver** that wires them together. Create the file inside the container (we use `nano`, a simple editor):

```
hadoop@a1b2c3:~$ nano WordCount.java
```

Paste the following, then save with `Ctrl+O`, Enter, and exit with `Ctrl+X`.

```
WordCount.java
```

```

import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

    // ----- MAPPER: (offset, line) -> (word, 1) -----
    public static class TokenizerMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
            StringTokenizer itr = new StringTokenizer(value.toString());
            while (itr.hasMoreTokens()) {
                word.set(itr.nextToken().toLowerCase());
                context.write(word, one);          // emit (word, 1)
            }
        }
    }

    // ----- REDUCER: (word, [1,1,1,...]) -> (word, sum) -----
    public static class IntSumReducer
        extends Reducer<Text, IntWritable, Text, IntWritable> {

        private IntWritable result = new IntWritable();

        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();                  // add up all the 1s
            }
            result.set(sum);
            context.write(key, result);            // emit (word, total)
        }
    }

    // ----- DRIVER: configure and launch the job -----
    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "word count");
        job.setJarByClass(WordCount.class);
        job.setMapperClass(TokenizerMapper.class);
        job.setCombinerClass(IntSumReducer.class); // combiner (see below)
        job.setReducerClass(IntSumReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
    }
}

```

```

        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}

```

Line-by-line: what each part does

Part	What it does
TokenizerMapper	Receives one line at a time (the value; the key is the line's byte offset). Splits the line into words with <code>StringTokenizer</code> , lower-cases each, and emits <code>(word, 1)</code> . This is K1,V1 → list(K2,V2) from your Unit 3 types slide.
<code>context.write(word, one)</code>	The map output. Every single word occurrence produces one pair — so a common word produces thousands of <code>(word, 1)</code> pairs.
Shuffle & sort	<i>You write no code for this.</i> Hadoop automatically groups all pairs with the same key, so the reducer receives <code>(word, [1,1,1,...])</code> . This is the shuffle from your Unit 3 centrepiece diagram.
IntSumReducer	Receives a word and the list of all its 1s, sums them, and emits <code>(word, total)</code> .
<code>setCombinerClass</code>	Runs the reducer logic <i>on the map side</i> first, collapsing many <code>(the,1)</code> into one <code>(the, 45)</code> before the shuffle — less network traffic. This is the combiner from your slides.

Execution & Expected Output

1 Create some input text

```

hadoop@alb2c3:~$ mkdir input
hadoop@alb2c3:~$ echo "the quick brown fox the lazy dog the fox" > input/file1.txt
hadoop@alb2c3:~$ echo "a quick dog and a brown fox jumped" > input/file2.txt

```

2 Compile the program

```

hadoop@alb2c3:~$ export HADOOP_CLASSPATH=$(hadoop classpath)
hadoop@alb2c3:~$ mkdir classes
hadoop@alb2c3:~$ javac -classpath $HADOOP_CLASSPATH -d classes WordCount.java
hadoop@alb2c3:~$ jar -cvf wordcount.jar -C classes .

```

The first line puts all Hadoop libraries on the compile path. `javac` compiles the source into `classes/`, and `jar` packages them into `wordcount.jar` that Hadoop can run.

3 Run the job

```

hadoop@alb2c3:~$ hadoop jar wordcount.jar WordCount input output

```

Watch the log output scroll past. Look for these lines — they are the stages from your slides made real:


```
map 0% reduce 0%
map 100% reduce 0%    <- all map tasks done
map 100% reduce 100%  <- reduce task done
Map-Reduce Framework
  Map input records=2
  Map output records=17    <- 17 (word,1) pairs emitted
  Combine input records=17
  Combine output records=11 <- combiner shrank 17 down to 11!
  Reduce input records=11
  Reduce output records=11
```

CHECKPOINT — SEE THE COMBINER WORKING

Notice **Combine input records=17** but **Combine output records=11**. The combiner merged duplicate words on the map side *before* the shuffle — exactly the traffic reduction your Unit 2/3 slides describe. Fewer records crossed to the reducer.

4 View the results

```
hadoop@a1b2c3:~$ cat output/part-r-00000
```

```
a      2
and    1
brown  2
dog    2
fox    3
jumped 1
lazy   1
quick  2
the    3
```

Every word with its total count, sorted alphabetically (the sort is free — it's part of the shuffle).

RE-RUNNING THE JOB

Hadoop refuses to overwrite an existing output folder (the “output directory must not already exist” rule from your Unit 3 slides!). To run again, delete it first: `rm -r output`, or use a new output folder name.

Experiment: turn the combiner off

Comment out the `job.setCombinerClass(...)` line, recompile, and re-run. You will now see **Combine input records=0** and the reducer receives all 17 records instead of 11. This is the clearest possible demonstration of what a combiner buys you.

5 Clean up when finished

```
hadoop@a1b2c3:~$ exit
PS> docker rm -f hadoop
```

Lab 2 · HDFS File Operations with the Java FileSystem API

Write, read, seek, and list files on HDFS from Java code

UNIT 2

JAVA

Aim

Use the Hadoop **FileSystem API** — the exact classes from your Unit 2 “Java Interface” slides — to interact with HDFS programmatically. You will `create()` a file, `open()` and `seek()` within it, and list a directory with `listStatus()` and `globStatus()`. This shows the write-once, seekable-read design of HDFS in action.

Requirements

- Docker Desktop running
- The `apache/hadoop` image running a single-node HDFS

LOCAL MODE VS. REAL HDFS

Lab 1 used Hadoop's local mode (no HDFS daemons). This lab starts an **actual HDFS** — a NameNode and a DataNode — so you can see files being stored and listed on a real distributed filesystem, even though it's one node.

Setup

1 Start single-node HDFS with Docker Compose

Back in Windows PowerShell, create the lab folder and a compose file:

```
PS> cd C:\bda-labs
PS> mkdir lab2-hdfs
PS> cd lab2-hdfs
PS> notepad docker-compose.yml
```

Paste this into Notepad and save:

```
docker-compose.yml
```

```

services:
  namenode:
    image: apache/hadoop:3.3.6
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - "9870:9870"
    env_file: ./config
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"
  datanode:
    image: apache/hadoop:3.3.6
    command: ["hdfs", "datanode"]
    env_file: ./config

```

And a small `config` file in the same folder (create it with `notepad config`):

config

```

CORE-SITE.XML_fs.default.name=hdfs://namenode
CORE-SITE.XML_fs.defaultFS=hdfs://namenode
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=1

```

2 Launch and format

```

PS> docker compose up -d
# wait ~15 seconds for the NameNode and DataNode to start

PS> docker compose exec namenode bash
hadoop@namenode:~$ hdfs dfs -mkdir -p /user/student
hadoop@namenode:~$ hdfs dfs -ls /user
# drwxr-xr-x ... /user/student

```

CHECKPOINT

`hdfs dfs -ls /user` shows the `student` directory you just created on HDFS. You can also open <http://localhost:9870> in your browser to see the NameNode web UI. HDFS is running.

The Program

Create the Java program inside the `namenode` container:

```
hadoop@namenode:~$ nano HdfsDemo.java
```

HdfsDemo.java

```

import java.net.URI;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.FileStatus;
import org.apache.hadoop.fs.FileSystem;
import org.apache.hadoop.fs.FSDataInputStream;
import org.apache.hadoop.fs.FSDataOutputStream;
import org.apache.hadoop.fs.Path;

public class HdfsDemo {
    public static void main(String[] args) throws Exception {

        String uri = "hdfs://namenode:8020";
        Configuration conf = new Configuration();
        FileSystem fs = FileSystem.get(URI.create(uri), conf);

        Path file = new Path("/user/student/quangle.txt");

        // ----- WRITE: create() gives an output stream -----
        FSDataOutputStream out = fs.create(file);
        out.writeBytes("On the top of the Crumpetty Tree\n"); // offset 0
        out.writeBytes("The Quangle Wangle sat\n"); // offset 33
        out.close();
        System.out.println("Wrote file: " + file);

        // ----- READ + SEEK: open() gives a seekable stream -----
        FSDataInputStream in = fs.open(file);
        byte[] buffer = new byte[32];
        in.read(buffer);
        System.out.println("From start : " + new String(buffer).trim());

        in.seek(33); // jump to absolute byte 33
        byte[] buf2 = new byte[32];
        in.read(buf2);
        System.out.println("After seek : " + new String(buf2).trim());
        in.close();

        // ----- METADATA: getFileStatus -----
        FileStatus st = fs.getFileStatus(file);
        System.out.println("Length : " + st.getLen() + " bytes");
        System.out.println("Replication: " + st.getReplication());

        // ----- LIST: listStatus on a directory -----
        System.out.println("--- listStatus /user/student ---");
        for (FileStatus s : fs.listStatus(new Path("/user/student"))) {
            System.out.println(" " + s.getPath().getName());
        }

        // ----- GLOB: wildcard matching -----
        System.out.println("--- globStatus *.txt ---");
        for (FileStatus s : fs.globStatus(new Path("/user/student/*.txt"))) {
            System.out.println(" " + s.getPath().getName());
        }

        fs.close();
    }
}

```

What each call demonstrates

Call	Unit 2 concept it shows
<code>FileSystem.get(uri, conf)</code>	Get a handle to HDFS. The URI scheme <code>hdfs://</code> selects the right filesystem — the “filesystems by scheme” idea.
<code>fs.create(file)</code>	Returns an <code>FSDDataOutputStream</code> . Writing is sequential only — no seeking on write (the write-once design).
<code>fs.open(file) + in.seek(33)</code>	Returns an <code>FSDDataInputStream</code> which supports random access . <code>seek()</code> jumps to any absolute byte position — reading is seekable, writing is not.
<code>getFileStatus</code>	Metadata: length, replication factor, and more — the <code>FileStatus</code> object from your slides.
<code>listStatus / globStatus</code>	Directory listing, and wildcard pattern matching (<code>*.txt</code>) — the four path/listing methods.

Execution & Expected Output

1 Compile and package

```
hadoop@namenode:~$ export HADOOP_CLASSPATH=$(hadoop classpath)
hadoop@namenode:~$ mkdir classes
hadoop@namenode:~$ javac -classpath $HADOOP_CLASSPATH -d classes HdfsDemo.java
hadoop@namenode:~$ jar -cvf hdfsdemo.jar -C classes .
```

2 Run it

```
hadoop@namenode:~$ hadoop jar hdfsdemo.jar HdfsDemo
```

```
Wrote file: /user/student/quangle.txt
From start : On the top of the Crumpey Tree
After seek : The Quangle Wangle sat
Length      : 56 bytes
Replication: 1
--- listStatus /user/student ---
    quangle.txt
--- globStatus *.txt ---
    quangle.txt
```

CHECKPOINT — SEEK IN ACTION

The line “**After seek**” printed the *second* line of the file, even though we opened the stream once.

`seek(33)` jumped straight to byte 33 — that is HDFS's seekable random-access read. Now confirm the file really lives on HDFS: `hdfs dfs -cat /user/student/quangle.txt` .

3 Add a second file and re-run the glob

```
hadoop@namenode:~$ echo "hello hdfs" | hdfs dfs -put - /user/student/notes.txt
hadoop@namenode:~$ hadoop jar hdfsdemo.jar HdfsDemo
```

The `listStatus` and `globStatus` sections now show *both* files — you added one from the command line and the Java program saw it immediately.

IF YOU RE-RUN AND IT ERRORS ON CREATE

`create()` overwrites by default, so re-running is fine here. If you ever see a “file already exists” error in other programs, that’s the write-once protection — delete first with `hdfs dfs -rm /path`.

4 Clean up

```
hadoop@namenode:~$ exit
PS> docker compose down
```

`docker compose down` stops and removes both the NameNode and DataNode containers in one command.

Lab 3 · Kafka Producer & Consumer

Write keyed messages to a topic and read them back with the poll loop

UNITS 4–5

JAVA

PYTHON

Aim

Run a single-broker Kafka cluster, create a topic with several partitions, then build a **producer** that sends keyed messages and a **consumer** that reads them with `poll()`. You will see key-based partitioning (Unit 4) and the consumer offset/poll loop (Unit 5) working together. Both Java and Python versions are given — pick whichever your class prefers.

Requirements

- Docker Desktop running
- The official `apache/kafka` image (KRaft mode — no ZooKeeper needed)

ONE BROKER IS ENOUGH

Your slides describe multi-broker clusters, but a single broker demonstrates producers, consumers, topics, partitions, keys, and offsets perfectly. The modern `apache/kafka` image runs in **KRaft mode**, so there is no separate ZooKeeper container to manage.

Setup

1 Start a Kafka broker

```
PS> cd C:\bda-labs
PS> mkdir lab3-kafka
PS> cd lab3-kafka

PS> docker run -d --name kafka -p 9092:9092 apache/kafka:3.7.0
```

The `-p 9092:9092` maps the broker's port to your Windows machine, so programs running *inside* the container reach it at `localhost:9092`.

2 Create a topic with 3 partitions

```
PS> docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --create --topic orders --partitions 3 --replication-factor 1 \
  --bootstrap-server localhost:9092

# Created topic orders.

PS> docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --describe --topic orders --bootstrap-server localhost:9092
```

CHECKPOINT

The `--describe` output shows **PartitionCount: 3** with three lines, one per partition (0, 1, 2), each with a Leader. Your topic exists with three partitions — enough to see key-based routing.

The Program — Python version

Python is the quickest to run. We install the client and write two short scripts. Open a shell in the container and set up Python:

```
PS> docker exec -it kafka bash
$ pip install kafka-python-ng      # the maintained Kafka client
```

producer.py

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    key_serializer=lambda k: k.encode('utf-8'),
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

for i in range(9):
    key = f"customer-{i % 3}"          # 3 distinct keys
    value = {"order_id": i, "item": f"widget-{i}"}
    # send returns a Future; .get() waits and returns metadata
    meta = producer.send('orders', key=key, value=value).get()
    print(f"key={key:12s} -> partition {meta.partition}, offset {meta.offset}")

producer.flush()
producer.close()
```

consumer.py

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    'orders',
    bootstrap_servers='localhost:9092',
    group_id='demo-group',
    auto_offset_reset='earliest',      # read from the start
    key_deserializer=lambda k: k.decode('utf-8'),
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

print("Polling for messages (Ctrl-C to stop)...")
for record in consumer:               # this is the poll loop
    print(f"partition={record.partition} offset={record.offset} "
          f"key={record.key} value={record.value}")
```


How this maps to your slides

Code	Concept
<code>key_serializer / value_serializer</code>	The three mandatory producer properties (Unit 4) — Kafka stores bytes, so objects must be serialized.
<code>key = customer-{i%3}</code>	Only 3 distinct keys. Kafka hashes the key to pick a partition, so all messages with the same key land on the same partition (Unit 4 partitioning).
<code>.send(...).get()</code>	Synchronous send — waits for the broker's <code>RecordMetadata</code> (partition + offset), the send-path from Unit 4.
<code>group_id + auto_offset_reset</code>	Consumer group and “start from earliest” (Unit 5 config).
<code>for record in consumer</code>	The poll loop — the heart of every consumer (Unit 5). Each record carries partition, offset, key, value.

Execution & Expected Output (Python)

You need **two shells into the same container**. Open a second PowerShell window and run `docker exec -it kafka bash` in it. Start the consumer first (so it's waiting), then run the producer in the other shell.

1 Shell A — start the consumer

```
$ python consumer.py
Polling for messages (Ctrl-C to stop)...
```

2 Shell B — run the producer

```
$ python producer.py
key=customer-0 -> partition 2, offset 0
key=customer-1 -> partition 0, offset 0
key=customer-2 -> partition 1, offset 0
key=customer-0 -> partition 2, offset 1
key=customer-1 -> partition 0, offset 1
key=customer-2 -> partition 1, offset 1
key=customer-0 -> partition 2, offset 2
key=customer-1 -> partition 0, offset 2
key=customer-2 -> partition 1, offset 2
```

CHECKPOINT — KEY-BASED PARTITIONING

Look at the producer output: **every customer-0 went to the same partition**, every `customer-1` to another, every `customer-2` to a third. The key's hash decides the partition, and it's consistent — exactly the Unit 4 guarantee. The offsets increase within each partition independently.

Back in Shell A, the consumer prints all nine records as it polls them — with their partition, offset, key, and value. Press `Ctrl+C` to stop it.

The Program — Java version

If your class works in Java, use these instead. They need the Kafka client JARs; the container already has them under `/opt/kafka/libs`.

SimpleProducer.java

```
import org.apache.kafka.clients.producer.*;
import java.util.Properties;

public class SimpleProducer {
    public static void main(String[] args) throws Exception {
        Properties p = new Properties();
        p.put("bootstrap.servers", "localhost:9092");
        p.put("key.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");
        p.put("value.serializer",
            "org.apache.kafka.common.serialization.StringSerializer");

        KafkaProducer<String,String> producer = new KafkaProducer<>(p);

        for (int i = 0; i < 9; i++) {
            String key = "customer-" + (i % 3);
            String value = "{\"order_id\":" + i + "}";
            ProducerRecord<String,String> rec =
                new ProducerRecord<>("orders", key, value);
            RecordMetadata m = producer.send(rec).get(); // sync send
            System.out.printf("key=%-12s -> partition %d, offset %d\n",
                key, m.partition(), m.offset());
        }
        producer.close();
    }
}
```

SimpleConsumer.java

```

import org.apache.kafka.clients.consumer.*;
import java.time.Duration;
import java.util.Collections;
import java.util.Properties;

public class SimpleConsumer {
    public static void main(String[] args) {
        Properties p = new Properties();
        p.put("bootstrap.servers", "localhost:9092");
        p.put("group.id", "demo-group");
        p.put("auto.offset.reset", "earliest");
        p.put("key.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");
        p.put("value.deserializer",
            "org.apache.kafka.common.serialization.StringDeserializer");

        KafkaConsumer<String,String> consumer = new KafkaConsumer<>(p);
        consumer.subscribe(Collections.singletonList("orders"));

        System.out.println("Polling... (Ctrl-C to stop)");
        while (true) {
            // the poll loop
            ConsumerRecords<String,String> records =
                consumer.poll(Duration.ofMillis(500));
            for (ConsumerRecord<String,String> r : records) {
                System.out.printf("partition=%d offset=%d key=%s value=%s%n",
                    r.partition(), r.offset(), r.key(), r.value());
            }
        }
    }
}

```

1 Compile and run (inside the container)

```

$ export CP="/opt/kafka/libs/*"
$ javac -cp "$CP" SimpleProducer.java SimpleConsumer.java

# Shell A:
$ java -cp ".:$CP" SimpleConsumer

# Shell B:
$ java -cp ".:$CP" SimpleProducer

```

The output is identical to the Python version — same key-to-partition mapping, same offsets. That's the point: the concepts are language-independent.

2 Clean up

```

$ exit
PS> docker rm -f kafka

```

KEEP THE BROKER FOR LAB 4

Lab 4 builds directly on this broker and the `orders` topic. If you're going straight on, **don't remove the container yet** — skip the cleanup step and continue to Lab 4.

Lab 4 · Consumer-Group Scaling & Rebalance

Watch partitions split across consumers — and rebalance live

UNIT 5

JAVA

PYTHON

Aim

This lab turns the group-scaling diagrams (Figs 4-1 → 4-4) from your Unit 5 slides into something you literally watch happen. You will run the *same consumer* in several terminals with the same `group.id`, and observe the broker split the partitions among them — then trigger a **rebalance** by adding and killing consumers.

Requirements

- The Kafka broker and `orders` topic from Lab 3 still running (`docker ps` should list `kafka`)
- If you removed it, re-run the Lab 3 setup (start broker + create 3-partition topic)

Setup

1 A continuous producer

We want a steady stream so you can see who's consuming what. Create a producer that sends one message per second forever. Open a shell: `docker exec -it kafka bash`.

stream.py

```
from kafka import KafkaProducer
import time, json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    key_serializer=lambda k: k.encode('utf-8'),
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

i = 0
while True:
    key = f"customer-{i % 6}" # spread across partitions
    producer.send('orders', key=key, value={"n": i})
    print(f"sent n={i} key={key}")
    i += 1
    time.sleep(1)
```

2 A labelled consumer

We reuse Lab 3's consumer but add a name tag so you can tell the instances apart.

group_consumer.py

```

from kafka import KafkaConsumer
import json, sys

name = sys.argv[1] if len(sys.argv) > 1 else "C?"

consumer = KafkaConsumer(
    'orders',
    bootstrap_servers='localhost:9092',
    group_id='scaling-group',          # SAME group for all instances
    auto_offset_reset='latest',
    key_deserializer=lambda k: k.decode('utf-8'),
    value_deserializer=lambda v: json.loads(v.decode('utf-8'))
)

print(f"[{name}] started, waiting for assignment...")
for record in consumer:
    print(f"[{name}] partition={record.partition} "
          f"offset={record.offset} value={record.value}")

```

THE KEY IDEA

Every consumer uses the **same group.id** (`scaling-group`). That makes them one group, so Kafka splits the topic's 3 partitions among them. The command-line name (`C1` , `C2` ...) is just so *you* can see which instance gets which partitions.

Execution & Expected Output

You'll use **up to four PowerShell windows**, each running `docker exec -it kafka bash` . Label them mentally: Producer, C1, C2, C3.

1 Start the producer (window 1)

```

$ python stream.py
sent n=0 key=customer-0
sent n=1 key=customer-1
...

```

2 Start ONE consumer (window 2)

```

$ python group_consumer.py C1
[C1] started, waiting for assignment...
[C1] partition=0 offset=15 value={'n': 30}
[C1] partition=1 offset=12 value={'n': 31}
[C1] partition=2 offset=14 value={'n': 32}

```

C1 alone reads all 3 partitions — this is Figure 4-1 from your slides.

3 Start a SECOND consumer (window 3)

```
$ python group_consumer.py C2
[C2] started, waiting for assignment...
[C2] partition=2 offset=20 value={'n': 45}
```

Watch C1's window — it **stops reading some partitions** the moment C2 joins. A **rebalance** just happened. Now C1 handles two partitions and C2 handles one (Figure 4-2).

CHECKPOINT — YOU JUST SAW A REBALANCE

When C2 joined, the broker's group coordinator reassigned partitions so the load splits. This is the “add a consumer to scale” behaviour from Unit 5, live. No partition is ever read by two consumers in the same group at once.

4 Start a THIRD consumer (window 4)

```
$ python group_consumer.py C3
```

Now each of C1, C2, C3 owns **exactly one** of the 3 partitions — maximum parallelism (Figure 4-3). Each window shows only its one partition number.

5 Start a FOURTH consumer — the idle one

```
$ python group_consumer.py C4
[C4] started, waiting for assignment...
(nothing else – C4 stays silent)
```

With only 3 partitions, the 4th consumer gets **nothing** — it sits idle (Figure 4-4). This is why partition count is the ceiling on how far a group scales.

6 Kill a consumer — watch recovery

In C1's window, press `Ctrl+C`. Within a few seconds another consumer (including the previously-idle C4) picks up C1's orphaned partition — a rebalance triggered by a consumer *leaving*. The group heals itself with no data lost.

CHECKPOINT — THE WHOLE UNIT 5 SCALING STORY, LIVE

You have now watched: one consumer take everything → adding consumers split the load → an extra consumer sit idle → a dead consumer's partitions get reassigned. That is Figures 4-1 through 4-4 plus the rebalance mechanism, all in real terminals.

Java version

The same demo works with the Lab 3 `SimpleConsumer.java` — just change the `group.id` to `"scaling-group"` and run several copies in separate shells. To tell them apart, add a name argument and print it,

mirroring `group_consumer.py`. The rebalance behaviour is identical; it's a broker-side mechanism, not a client-language feature.

7 Clean up

```
# Ctrl-C every consumer and the producer, then exit each shell  
PS> docker rm -f kafka
```

KEEP THE BROKER FOR LAB 5 IF CONTINUING

Lab 5 also uses a Kafka broker. If you're going straight on, leave the container running and reuse it.

Lab 5 · Avro Serialization Round-Trip

Produce and consume schema-based records — why schemas beat hand-rolled formats

UNITS 2 & 4

JAVA

PYTHON

Aim

Define an **Avro schema** for a `Customer` record, serialize objects with it, send them through Kafka, and deserialize them back into objects on the other side. This ties the Unit 2 Avro/serialization material to the Unit 4 producer serializers, and shows concretely why a shared schema is safer than a custom byte-format.

Requirements

- Docker Desktop running
- A Kafka broker (reuse Lab 3/4's, or start a fresh one)

AVRO WITHOUT A SCHEMA REGISTRY

Your Unit 4 slides show Avro *with* a Schema Registry. For a first lab we keep it simpler: the schema lives in a local `.avsc` file that both sides read. This still demonstrates the core idea — **a schema is a contract** shared by producer and consumer — without running an extra Registry service. (A note at the end explains what the Registry adds.)

Setup

1 Ensure a broker and topic exist

```
PS> docker run -d --name kafka -p 9092:9092 apache/kafka:3.7.0

PS> docker exec -it kafka /opt/kafka/bin/kafka-topics.sh \
  --create --topic customers --partitions 1 --replication-factor 1 \
  --bootstrap-server localhost:9092
```

(If your Lab 3/4 broker is still running, skip the first line and just create the `customers` topic.)

2 Install the Avro + Kafka Python libraries

```
PS> docker exec -it kafka bash
$ pip install kafka-python-ng avro
```

The Program — Python version

First, the schema — the shared contract. Create it inside the container:

```
customer.avsc
```



```
{
  "namespace": "bda.avro",
  "type": "record",
  "name": "Customer",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "name", "type": "string"},
    {"name": "email", "type": ["null", "string"], "default": null}
  ]
}
```

avro_producer.py

```
from kafka import KafkaProducer
import avro.schema, avro.io, io

schema = avro.schema.parse(open("customer.avsc").read())

def encode(record):
    buf = io.BytesIO()
    encoder = avro.io.BinaryEncoder(buf)
    writer = avro.io.DatumWriter(schema)
    writer.write(record, encoder)      # object -> Avro bytes
    return buf.getvalue()

producer = KafkaProducer(bootstrap_servers='localhost:9092')

customers = [
    {"id": 1, "name": "Asha", "email": "asha@x.com"},
    {"id": 2, "name": "Ravi", "email": None},
    {"id": 3, "name": "Meena", "email": "meena@x.com"},
]

for c in customers:
    producer.send('customers', value=encode(c))
    print(f"sent (Avro-encoded): {c}")

producer.flush()
producer.close()
```

avro_consumer.py

```

from kafka import KafkaConsumer
import avro.schema, avro.io, io

schema = avro.schema.parse(open("customer.avsc").read())

def decode(data):
    buf = io.BytesIO(data)
    decoder = avro.io.BinaryDecoder(buf)
    reader = avro.io.DatumReader(schema)
    return reader.read(decoder)          # Avro bytes -> object

consumer = KafkaConsumer(
    'customers',
    bootstrap_servers='localhost:9092',
    group_id='avro-group',
    auto_offset_reset='earliest'
)

print("Reading Avro records...")
for record in consumer:
    customer = decode(record.value)
    print(f"decoded object: id={customer['id']} "
          f"name={customer['name']} email={customer['email']}")

```

What this demonstrates

Piece	Concept
<code>customer.avsc</code>	The Avro schema in JSON (Unit 2). Both producer and consumer read the same file — the shared contract.
<code>email:</code> <code>["null","string"]</code>	A nullable field with a default — Avro's schema evolution support, so the format can grow without breaking readers.
<code>DatumWriter</code> / encode	Turns a Python object into compact Avro binary — the producer serializer role from Unit 4.
<code>DatumReader</code> / decode	Turns bytes back into an object on the consumer side — the deserializer must use a matching schema.

Execution & Expected Output (Python)

1 Shell A — run the consumer

```

$ python avro_consumer.py
Reading Avro records...

```

2 Shell B — run the producer

```
$ python avro_producer.py
sent (Avro-encoded): {'id': 1, 'name': 'Asha', 'email': 'asha@x.com'}
sent (Avro-encoded): {'id': 2, 'name': 'Ravi', 'email': None}
sent (Avro-encoded): {'id': 3, 'name': 'Meena', 'email': 'meena@x.com'}
```

Back in Shell A, the consumer decodes and prints the reconstructed objects:

```
decoded object: id=1 name=Asha email=asha@x.com
decoded object: id=2 name=Ravi email=None
decoded object: id=3 name=Meena email=meena@x.com
```

CHECKPOINT — THE ROUND-TRIP

An object became compact Avro bytes, travelled through Kafka, and came back as an object with the same fields — including the nullable `email`. The producer and consumer never exchanged code, only the *schema file*. That is the whole point: serialization is a contract, and the contract lives in the schema.

3 Prove why the schema matters

Peek at the raw bytes on the topic with the console consumer:

```
$ /opt/kafka/bin/kafka-console-consumer.sh --topic customers \
  --from-beginning --bootstrap-server localhost:9092
```

You'll see **unreadable binary** — Avro is compact, not human-readable. Without the schema, those bytes are meaningless. *With* the schema, both sides interpret them identically. Press `Ctrl+C` to stop.

Java version

The Java flow mirrors the Python one: use the `org.apache.avro` library to parse `customer.avsc`, a `GenericDatumWriter` to encode a `GenericRecord` into bytes for the producer's `value`, and a `GenericDatumReader` to decode on the consumer side. The producer/consumer plumbing is exactly Lab 3's Java code with a `ByteArraySerializer` / `ByteArrayDeserializer` for the value, and the Avro encode/decode wrapped around it.

WHAT A SCHEMA REGISTRY WOULD ADD

Here we shipped the schema as a file both sides already had. In production (your Unit 4 Fig 3-3), a **Schema Registry** stores schemas centrally: the producer registers the schema once and embeds only a small *schema id* in each message; the consumer fetches the schema by id. That lets schemas evolve safely across many teams without copying `.avsc` files around. The Confluent `cp-schema-registry` image adds this as one more container when you're ready.

4 Clean up everything

```
$ exit
PS> docker rm -f kafka
PS> docker ps          # confirm nothing is left running
```

YOU'RE DONE

Across five labs you have run MapReduce, driven HDFS from Java, produced and consumed Kafka messages, watched a consumer group scale and rebalance, and round-tripped Avro records — every one inside Docker, matching Units 2 through 5 of the course.